

CIS 371 Web Application Programming

ExpressJS & Cloud Functions

HTTP server in (Java|Type)Script



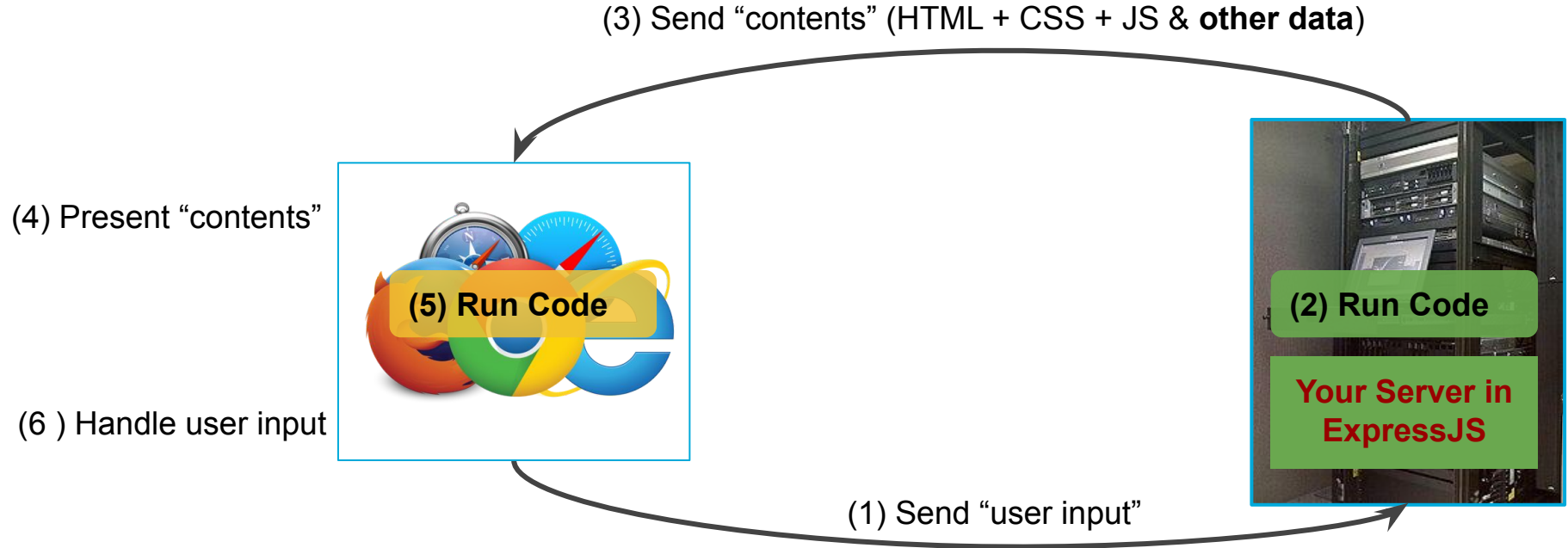
GRAND VALLEY
STATE UNIVERSITY®

Lecturer: **Dr. Yong Zhuang**

Topics

- What is ExpressJS
- Why need an app server
- Features provided
 - Router for HTTP methods (GET, POST, PUT, DELETE, ...)
 - Middleware
 - Query Parameter parsing
 - Payload parsing (via middleware)
- Prerequisite
 - HTTP Protocol: Request & Response

Web Client/Server Architecture



Why Use an App Server

- Enhanced Computing Power:
 - Client Side: Limited computing capabilities on user machines.
 - App Server: Typically hosted on robust external machines offering greater computing power.
- Security Considerations:
 - Code Visibility: Code sent to a client's browser can be inspected by the user, possibly revealing proprietary information.
 - Company Secrets: Using an app server can protect sensitive logic and data from direct client access.
- Be aware of extra network transport overhead
- Potential Use Cases:
 - Proxy Server: Bypass CORS restrictions when accessing certain web services.
 - Data Aggregation: Consolidate data from various sources like web clients and mobile clients.
 - High-End Computations: Ideal for tasks that require heavy computation, such as machine learning, computational simulations, and data mining.

Alternatives to ExpressJS



Setup

```
npm init -y  
npm i express # version 4.x
```

```
// Additional libraries for TypeScript dev  
npm i --save-dev typescript ts-node nodemon  
  
// Add type declaration files  
npm i --save-dev @types/express @types/node  
  
// Creates tsconfig.json  
tsc --init
```

tsconfig.json

```
{  
  "compilerOptions": {  
    "target": "es6",  
    "module": "commonjs",  
    "rootDir": "./",  
    "outDir": "./build",  
    "esModuleInterop": true,  
    "strict": true  
  }  
};
```

Hello World

```
import express, { Application, Request, Response } from "express";

const app: Application = express();
const PORT = process.env.PORT ?? 8000;

// Define GET endpoint(s)
app.get("/", (req: Request, res: Response) => {
  res.send("Hello World!");
});

app.listen(PORT, () => {
  console.log(`Server is listening to port ${PORT}`);
});
```

my-server.ts

```
// Launch the server
npx nodemon my-server.ts
```

Browser

<http://localhost:8000/>

Defining More Routes/EndPoints

```
// import lines not shown
app.get("/", (req: Request, res: Response) => {
  res.send("Hello World!");
});
app.get("/about", (req: Request, res: Response) => {
  res.send("Just a simple Express server");
});
app.post("/xyz", (req: Request, res: Response) => {
  res.send("Just a simple Express server");
});
app.put("/order/cancel/", (req: Request, res: Response) => {
  res.send("_____");
});
app.delete("/account", (req: Request, res: Response) => {
  res.send("_____");
});
```

my-server.ts

Browser

http://localhost:8000/

Browser

http://localhost:8000/about

*Can't invoke these from
browser omnibox*

Sending Responses

```
// METHOD: get, post, put, delete, and so on
app.METHOD("/path/goes/here", (req: Request, res: Response) => {
  // any file type, its content will be included in the response body
  res.download("file-name"); // Without MIME type
  res.type("image/jpg").download("mycat.jpg"); // OR with MIME type
});
```

Opt #1: file attachment

```
app.METHOD("/path/goes/here", (req: Request, res: Response) => {
  res.send("some text here"); // Opt-2: send text response
  res.type("text/plain").send("some text here"); // Opt-2a: with Content-Type
});
```

Opt #2: plain text

```
app.METHOD("/path/goes/here", (req: Request, res: Response) => {
  res.send("<P>Sample HTML response</p>"); // Opt-3: send HTML response
  res.type("text/html").send("<P>Sample HTML response</p>"); // Opt-3a: with Content-Type
});
```

Opt #3: HTML string

Sending Responses

```
app.METHOD("/path/goes/here", (req: Request, res: Response) => {  
  res.send({ shippingCost: 5.16, sameDay: false }); // Opt-4: send JSON response  
});
```

Opt #4: JSON

```
app.METHOD("/path/goes/here", (req: Request, res: Response) => {  
  // Opt-5: send JSON from a JS object  
  const my_resp_obj = { msg: "Hello World", hasEmoji: false };  
  res.json(my_resp_obj);  
});
```

Opt #5: JSON from object

```
app.METHOD("/path/goes/here", (req: Request, res: Response) => {  
  res.status(401).end(); // Opt-6: set HTTP status & an empty response  
});
```

Opt #6: empty response

Parsing Dynamic Path/Route Names

```
app.get("/user/:uname", (req: Request, res: Response) => {  
  const who = req.params.uname;  
  if (who == "___") res.send(`Option #1 here`);  
  else res.send(`Option #2 here`);  
});
```

Browser <http://localhost:8000/user/bob>

```
app.get("/search/:minPrice/:maxPrice", (req: Request, res: Response) => {  
  const low = req.params.minPrice;  
  const high = req.params.maxPrice;  
  res.send(`Price range ${low}-${high}`);  
});
```

Browser <http://localhost:8000/search/50/110>

Parsing Query Parameters (TypeScript types)

```
type Query2 = {  
  min: number;  
  max: number;  
};
```

Browser `http://localhost:8000/search?min=50&max=110`

```
app.get("/search", (req: Request<any, any, any, Query2>, res: Response) => {  
  res.send(`Price range ${req.query.min}-${req.query.max}`);  
});
```

Recall: Firebase is a Collection of Many Products

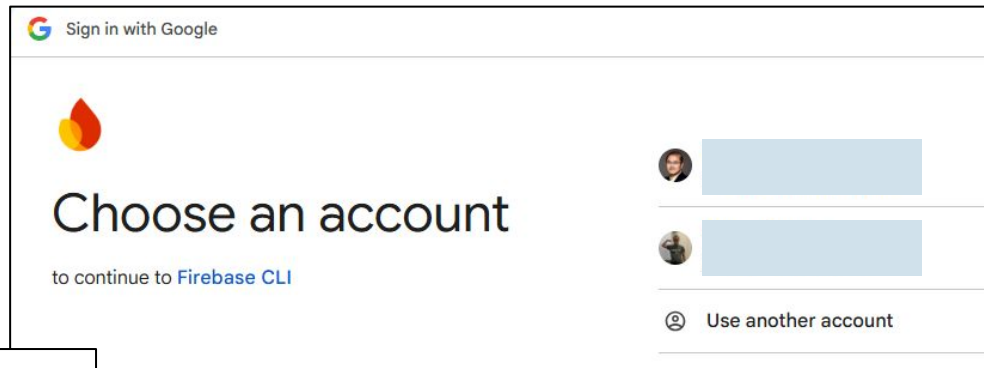
- Authentication
- Cloud Firestore
- Cloud Storage
- **Cloud Functions**
- ...

Project Setup & Initialization

```
node → ~/projects $ npm install -g firebase-tools
```



```
node → ~/projects $ firebase login
```



Woohoo!

Firebase CLI Login Successful

You are logged in to the Firebase Command-Line interface. You can immediately close this window and continue using the CLI.



Setup & Initialization

```
node →~/projects $ mkdir fire-functions  
node →~/projects $ cd fire-functions/
```

```
node →~/projects/fire-functions $ firebase init
```

```
#####  
##  
#####  
##  
#####
```

You're about to initialize a Firebase project in this directory:

`/home/node/projects/fire-functions`

? Which Firebase features do you want to set up for this directory? Press Space to select features, then Enter to confirm your choices.

- ☐ Data Connect: Set up a Firebase Data Connect service
- ☐ Firestore: Configure security rules and indexes files for Firestore
- ☐ Genkit: Setup a new Genkit project with Firebase
- ☒ **Functions: Configure a Cloud Functions directory and its files**
- ☐ App Hosting: Set up deployments for full-stack web apps (supports server-side rendering)
- ☐ Hosting: Set up deployments for static web apps
- ☐ Storage: Configure a security rules file for Cloud Storage

```
✓ What language would you like to use to write Cloud Functions? TypeScript  
✓ Do you want to use ESLint to catch probable bugs and enforce style? No  
✓ Wrote functions/package.json  
✓ Wrote functions/tsconfig.json  
✓ Wrote functions/src/index.ts  
✓ Wrote functions/.gitignore  
✓ Do you want to install dependencies with npm now? Yes
```

Setup & Initialization

```
> .agents
> .claude
✓ functions
> node_modules
✓ src
  | TS index.ts
  | .gitignore
  | {} package-lock.json
  | {} package.json
  | TS tsconfig.json
  | .firebaserc
  | .gitignore
  | firebase.json
  | } skills-lock.json
```

```
* See a full list of supported triggers at https://firebase.google.com/docs
*/

import {setGlobalOptions} from "firebase-functions";
import {onRequest} from "firebase-functions/https";
import * as logger from "firebase-functions/logger";

// Start writing functions
// https://firebase.google.com/docs/functions/typescript

// For cost control, you can set the maximum number of containers that can be
// running at the same time. This helps mitigate the impact of unexpected
// traffic spikes by instead downgrading performance. This limit is a
// per-function limit. You can override the limit for each function using the
// `maxInstances` option in the function's options, e.g.
// `onRequest({ maxInstances: 5 }, (req, res) => { ... })`.
// NOTE: setGlobalOptions does not apply to functions using the v1 API. v1
// functions should each use functions.runWith({ maxInstances: 10 }) instead.
// In the v1 API, each function can only serve one request per container, so
// this will be the maximum concurrent request count.
setGlobalOptions({ maxInstances: 10 });

// export const helloWorld = onRequest((request, response) => {
//   logger.info("Hello logs!", {structuredData: true});
//   response.send("Hello from Firebase!");
// });
```

index.ts

Install Firebase Emulators

```
node → ~/projects/fire-functions $ firebase init emulators
```

i Using project beverageshop-876a8 (BeverageShop) .

=== Emulators Setup

? Which Firebase emulators do you want to set up? Press Space to select emulators, then Enter to confirm your choices.

- ☐ App Hosting Emulator
- ☒ Authentication Emulator
- ☒ Functions Emulator
- > ☒ Firestore Emulator**
- ☐ Database Emulator
- ☐ Hosting Emulator
- ☒ Pub/Sub Emulator

=== Emulators Setup

✓ Which Firebase emulators do you want to set up? Press Space to select emulators, then Enter to confirm your choices.

✓ Which port do you want to use for the auth emulator? 9099

✓ Which port do you want to use for the functions emulator? 5001

✓ Which port do you want to use for the firestore emulator? 8080

✓ Which port do you want to use for the pubsub emulator? 8085

✓ Would you like to enable the Emulator UI? Yes

✓ Which port do you want to use for the Emulator UI (leave empty to use any available port)?

✓ Would you like to download the emulators now? Yes

i **firestore:** downloading cloud-firestore-emulator-v1.20.4.jar...

Progress: =====

i **pubsub:** downloading pubsub-emulator-0.8.29.zip...

Progress: =====

i **ui:** downloading ui-v1.15.0.zip...

=== Agent Skills Setup

If you are using an AI coding agent, Firebase Agent Skills make it an expert at Firebase.

Progress: =====

Firestore Emulators require JAVA

```
node → ~/projects/fire-functions $ sudo apt-get update
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9,671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5,412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [123 kB]
Fetched 10.0 MB in 9s (1,097 kB/s)
Reading package lists... Done
node → ~/projects/fire-functions $ sudo apt-get install -y openjdk-21-jdk
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```



```
node → ~/projects/fire-functions $ firebase emulators:start
```

✓ All emulators ready! It is now safe to connect your app.
i View Emulator UI at <http://127.0.0.1:4000/>

Firestore Emulator Suite

Remember this is a local environment. Visit the production version of [beveragehop-476a8](#) in the console.

Emulator overview

Authentication emulator Status: On Port number: 9099 Go to auth emulator	Firestore emulator Status: On Port number: 8080 Go to firestore emulator
Functions emulator Status: On Port number: 5001 Go to the logs page	Extensions emulator Status: On Port number: 5001 Go to extensions emulator



Emulator	Host:Port	View in Emulator UI
Authentication	127.0.0.1:9099	http://127.0.0.1:4000/auth
Functions	127.0.0.1:5001	http://127.0.0.1:4000/functions
Firestore	127.0.0.1:8080	http://127.0.0.1:4000/firestore
Pub/Sub	127.0.0.1:8085	n/a
Extensions	127.0.0.1:5001	http://127.0.0.1:4000/extensions

Firebase Emulators require JAVA

```
export const helloWorld = onRequest((request, response) => {  
  logger.info("Hello logs!", { structuredData: true });  
  response.send("Hello from Firebase!");  
});
```

index.ts



```
node → ~/projects/fire-functions/functions $ npm run build  
  
> build  
> tsc
```

<http://localhost:5001/<your-project-id>/us-central1/helloWorld>

⌵ | ID HPC Faculty Teaching YouTube Maps Gr
Hello from Firebase!

Firestore Emulators require JAVA

```
export const api = onRequest(async (req, res) => {  
  switch (req.method) {  
    case "GET":  
      res.send("GET request received!");  
      break;  
    case "POST":  
      const body = req.body;  
      res.send(body);  
      break;  
    case "DELETE":  
      res.send("DELETE request received!");  
      break;  
    default:  
      res.send("It was a default request!");  
  }  
});
```

index.ts